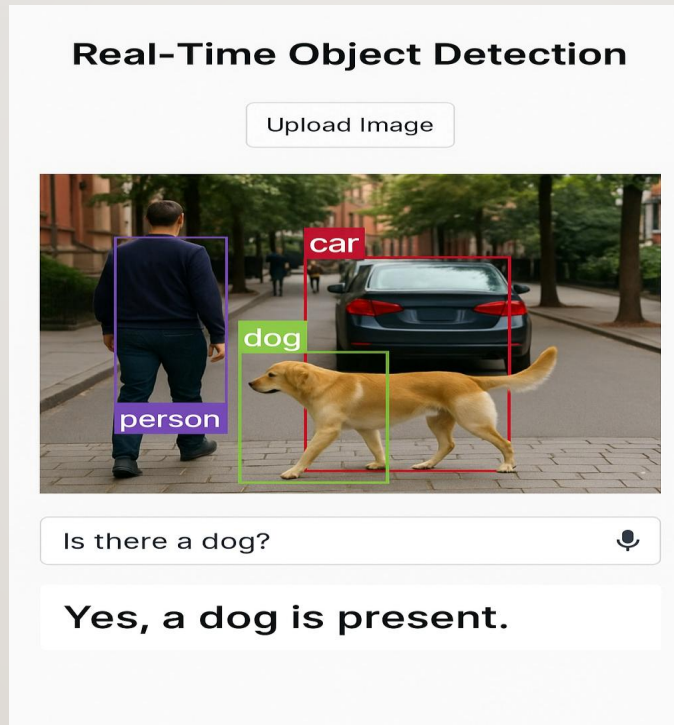


OBJECT DETECTION FROM AN IMAGE

INTERNAL GUIDE: MR. RAJESH



ABSTRACT

This project focuses on developing a user-friendly web-based application that performs real-time object detection on uploaded images using computer vision and machine learning. The core objective is to provide users with immediate visual feedback by identifying and labeling objects within an image through bounding boxes and class names. The application integrates advanced object detection algorithms like SSD (Single Shot MultiBox Detector), the project showcases the real-world utility in an accessible and interactive format. The front-end of the application is built using HTML, CSS, and JavaScript to ensure a responsive and interactive user experience. The back-end utilizes a Python-based web framework, such as Flask to handle image uploads, manage detection requests, and serve results. Object detection models are implemented using powerful machine learning libraries such as TensorFlow. The final product will be a functional, visually appealing web application that not only demonstrates the capabilities of object detection technologies but also provides a platform for extended experimentation in the field of computer vision.

INTRODUCTION

- Object detection plays a key role in various computer vision applications like autonomous vehicles, surveillance, and healthcare.
- Advances in AI and machine learning have significantly improved the accuracy and speed of object detection.
- Despite its potential, using these technologies often requires technical expertise.
- This project aims to make object detection accessible through a simple and intuitive web-based interface.
 - Users can upload images and receive real-time detection results without needing programming or ML knowledge.
- The goal is to bridge the gap between complex AI models and everyday users through an easy-to-use platform.

-EXISTING SYSTEMS

1. Google Cloud Vision API

Cloud-based ML service by Google that detects and labels objects in images using pre-trained deep learning models. Supports object localization with bounding boxes via REST API.

2. Amazon Rekognition

AWS service for image and video analysis. Provides real-time object, scene, and activity detection using deep neural networks. Easily integrates with AWS infrastructure.

3. YOLOv5 (by Ultralytics)

A fast and accurate object detection model implemented in PyTorch. Open-source and optimized for both real-time inference and high accuracy. Widely used in research and industry for its speed-performance balance.

-PROPOSED SYSTEMS

1. A web-based application for real-time object detection on uploaded images using the SSD model in TensorFlow.
2. Detects and labels multiple objects with bounding boxes and class names.
3. Built with Flask, OpenCV, HTML, CSS, and JavaScript for a responsive, full-stack experience.
4. New Feature: Users can ask if a specific object is present in the image (e.g., "Is there a dog?").
 - System searches detected objects and responds with Yes/No and visual highlight if found.
5. Enhances user interactivity by combining detection with intelligent object querying.
6. Voice-Based Object Query:
 - Voice Input: Users can ask through voice if a specific object (e.g., "Is there a cat?") is in the image.
 - The system:
 - Converts voice to text (using Web Speech API or SpeechRecognition in Python).
 - Searches detected objects for the queried item.
 - Responds using text-to-speech (e.g., "Yes, a cat is present") via browser's speech synthesis.

-TECHNOLOGIES USED

◇ Frontend:

- HTML5 – Structure and layout of the web interface.
- CSS3 – Styling and responsive design.
- JavaScript – Interactivity, image preview, and voice interaction.
- Web Speech API
 - Speech Recognition API – For capturing voice input (speech-to-text) directly from the user's microphone.
 - Speech Synthesis API – For generating voice responses directly in the browser, providing spoken feedback to the user.

◇ Backend:

- Python 3.x – Core programming language.
- Flask – Lightweight web framework for routing and server-side logic.

- OpenCV – Image processing and handling bounding boxes around detected objects.
- TensorFlow – Deep learning framework for loading and running the SSD model for object detection.
- NumPy – For numerical operations required during image processing.
- Pillow (PIL) – For image format conversion and manipulation.

Machine Learning / Computer Vision:

- SSD (Single Shot MultiBox Detector) – Pre-trained model for detecting multiple objects in images.
- TensorFlow Object Detection API – Used for integrating and running the SSD model in the backend.

-MODULES:

1. Image Upload Module

- Allows users to upload images through a simple web interface.

2. Object Detection Module

- Uses the SSD model in TensorFlow to detect and classify objects in the uploaded image.

3. Query Module (Voice/Text Input)

- Accepts user queries asking if a specific object is present, via voice or text.

4. Search & Response Module

- Compares detected objects with the query and returns a "Yes/No" response based on presence.

5. Voice Output Module

- Converts the system's answer into speech output using text-to-speech.

6. Frontend (Web Interface) Module

- Built using HTML, CSS, JavaScript for user interaction and displaying results.

7. Backend Module (Flask)

- Connects the frontend, handles image processing, object detection, and user queries.

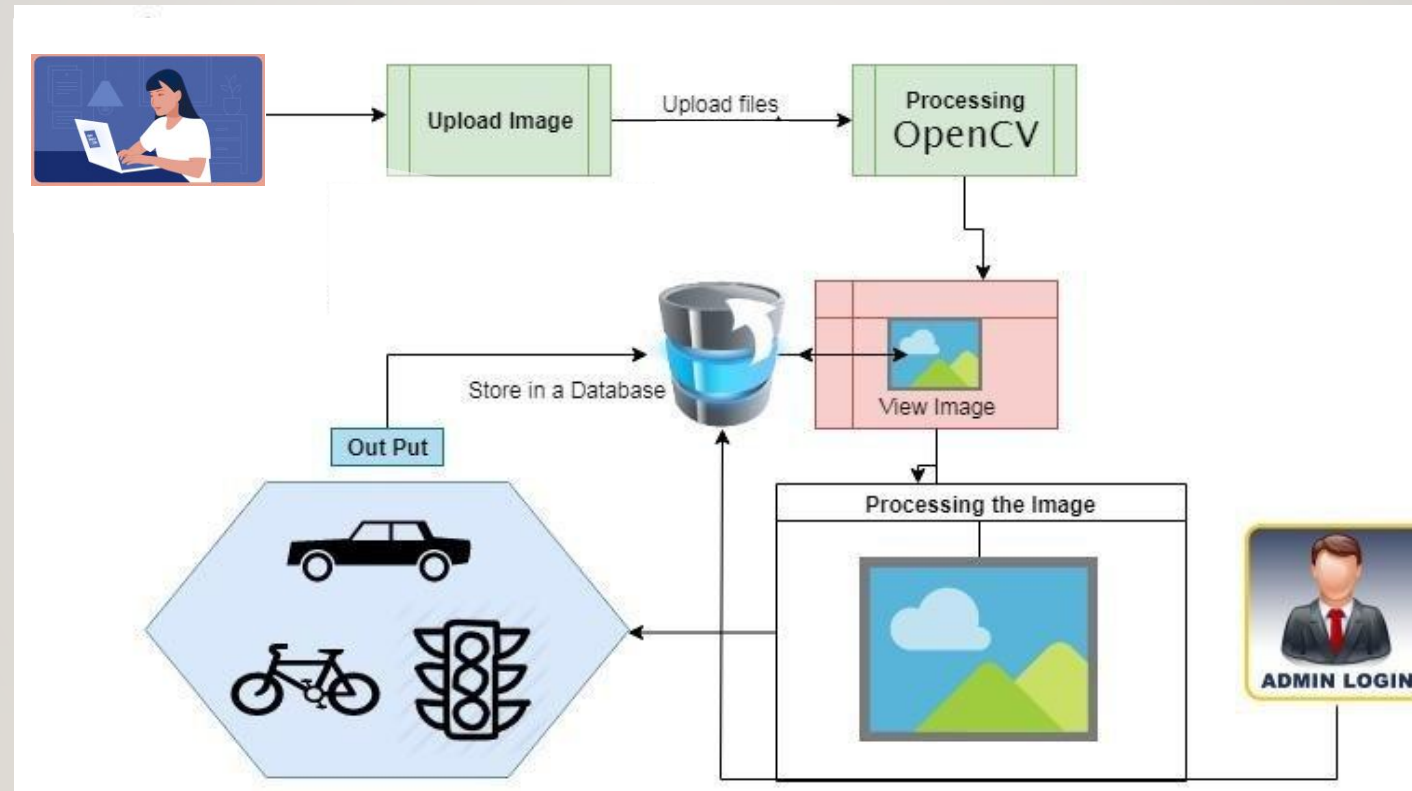
-SOFTWARE REQUIREMENTS

- Operating System: Compatible with Windows 10/11, Linux (Ubuntu), or macOS.
- Programming Language: Python 3.x for back-end development and model integration.
 - Web Framework: Flask is preferred for its simplicity and lightweight nature; Django can also be used.
- Frontend Technologies: HTML5, CSS3, and JavaScript for building a responsive user interface.
- Object Detection Framework: TensorFlow with the SSD (Single Shot MultiBox Detector) model using the TensorFlow Object Detection API.
 - Image Processing Library: OpenCV for handling image input/output and drawing bounding boxes.
- Voice Input Handling: Web Speech API (JavaScript) or Python Speech Recognition library to capture and convert spoken queries.
- Voice Output (Text-to-Speech): Web Speech Synthesis API or pyttsx 3 (Python) to provide spoken responses to the user.
- Browser: A modern web browser like Google Chrome, Firefox, or Microsoft Edge (latest version). Additional Libraries: Includes NumPy, Pillow, Werkzeug, and others needed to support image processing and model serving.

-HARDWARE REQUIREMENTS

- Processor: Minimum: Intel Core i5 (8th Gen) or AMD equivalent Recommended: Intel Core i7 or AMD Ryzen 7 for better performance.
- RAM: At least 8 GB is required, but 16 GB or more is recommended for smooth execution of model inference and server operations.
- Storage: Minimum 100 GB HDD Recommended: 256GB SSD for faster read/write and load times.
- Graphics Processing Unit (GPU): While the system can run on CPU, a CUDA-compatible NVIDIA GPU (e.g., GTX 1650 or higher) is recommended for faster object detection.
- Microphone: A built-in or external microphone is required for voice input.
- Speakers / Audio Output: Necessary for delivering voice-based responses through the system.
- Internet Connection: Required for downloading libraries, setting up dependencies, and loading pre-trained models during development.

SYSTEM ARCHITECTURE :



-UML DIAGRAMS

-CLASS DIAGRAM

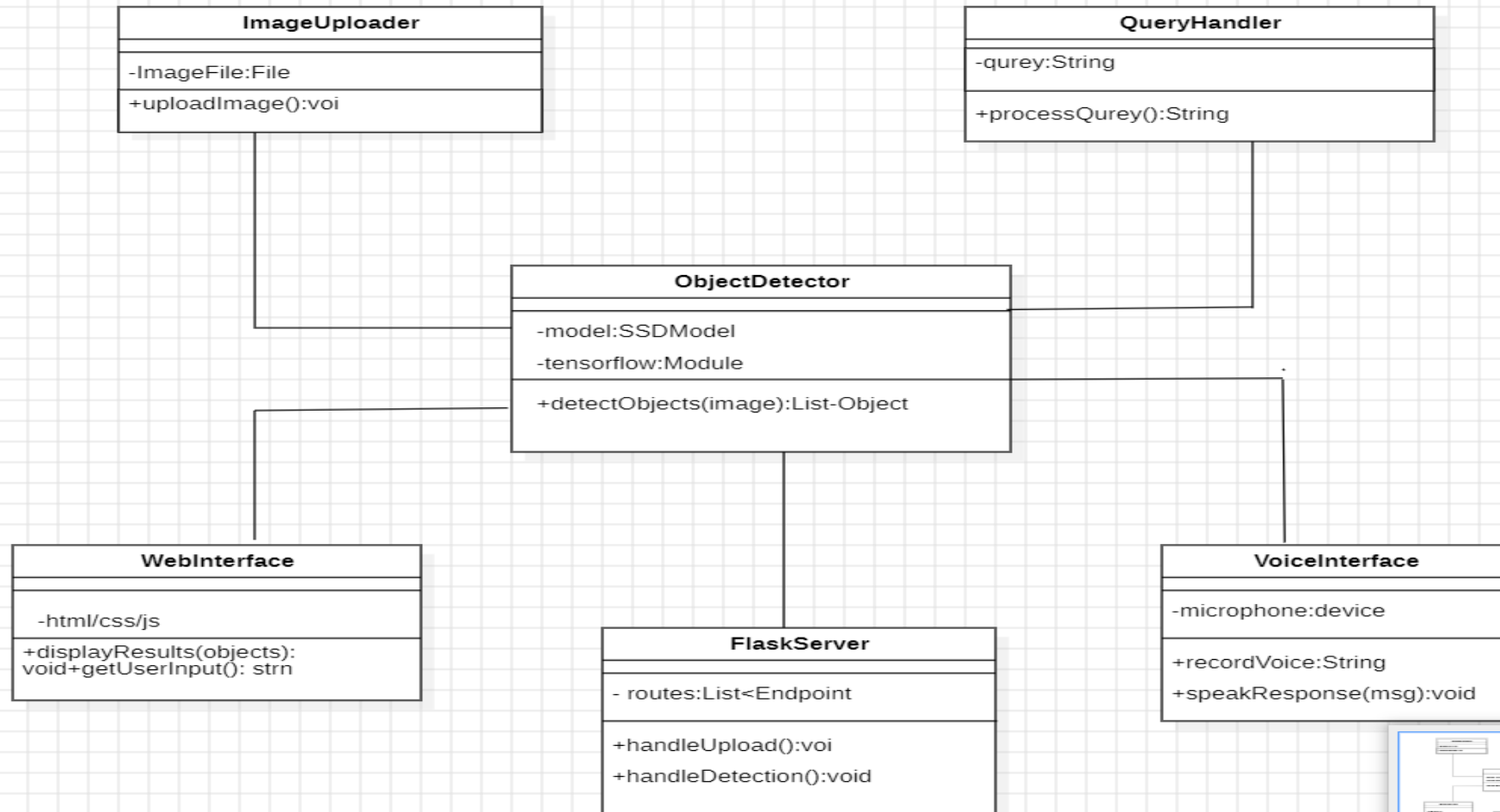
-USE CASE DIAGRAM

-SEQUENCE DIAGRAM

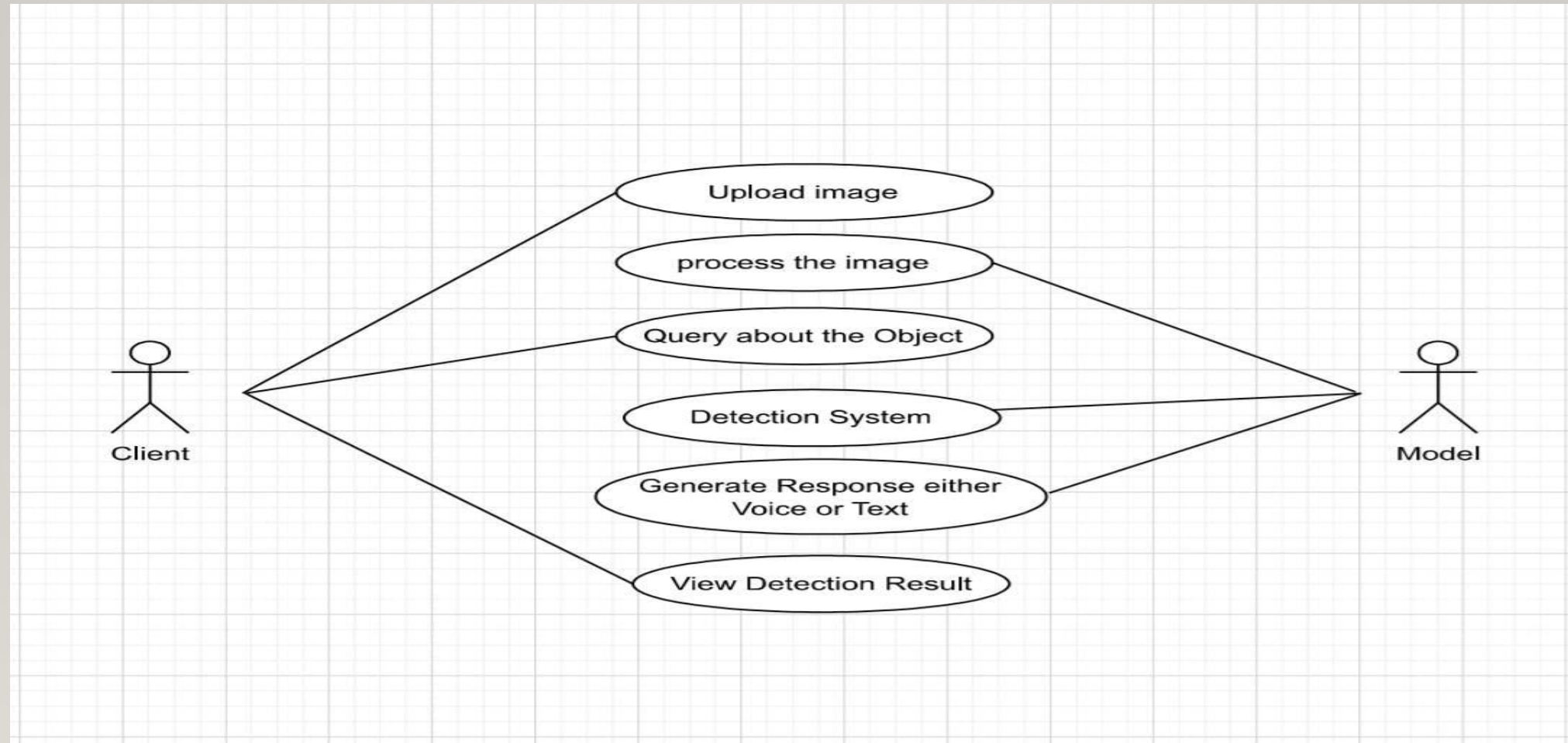
-ACTIVITY DIAGRAM

-WORK FLOW DIAGRAM

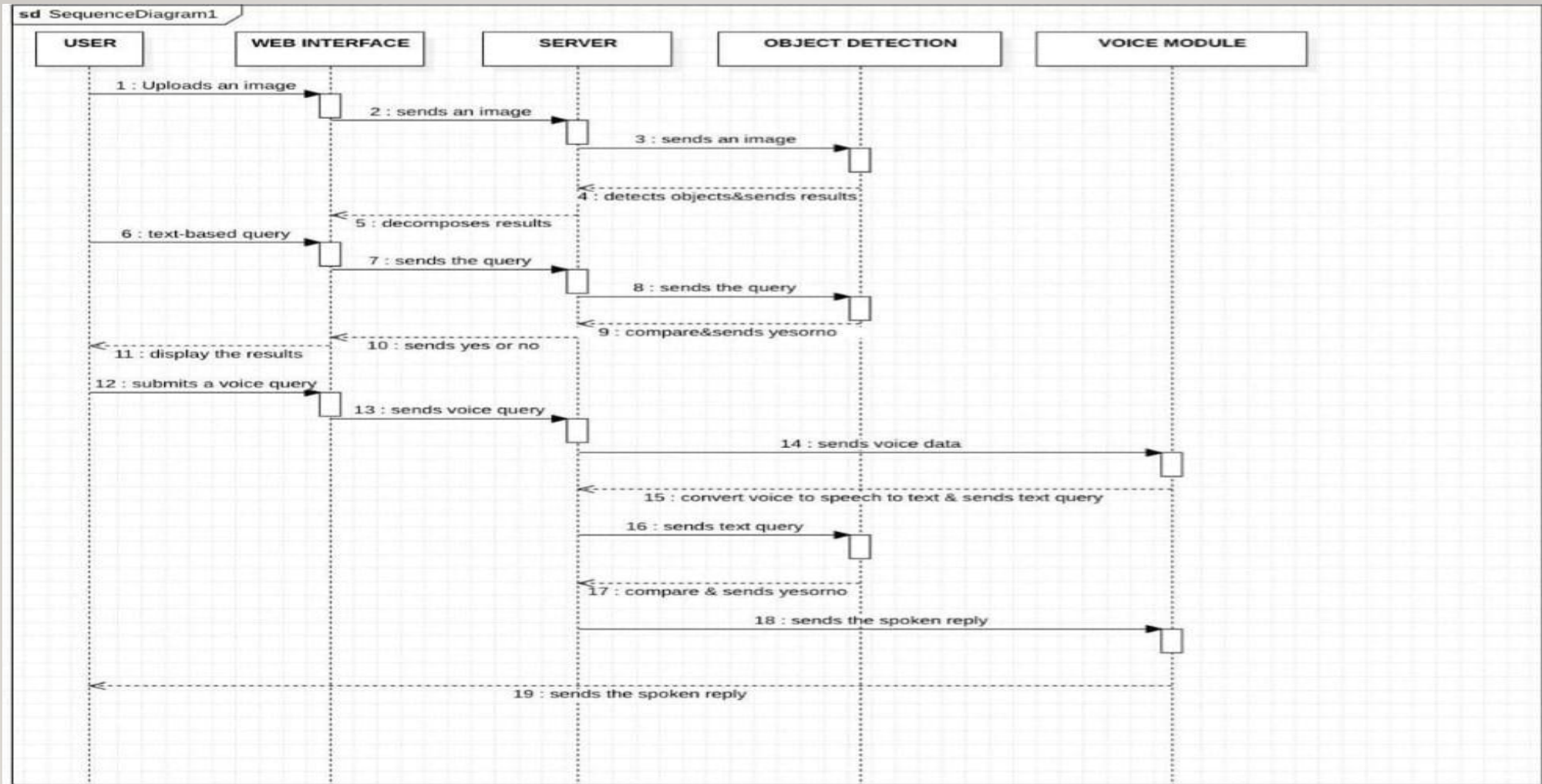
CLASS DIAGRAM:



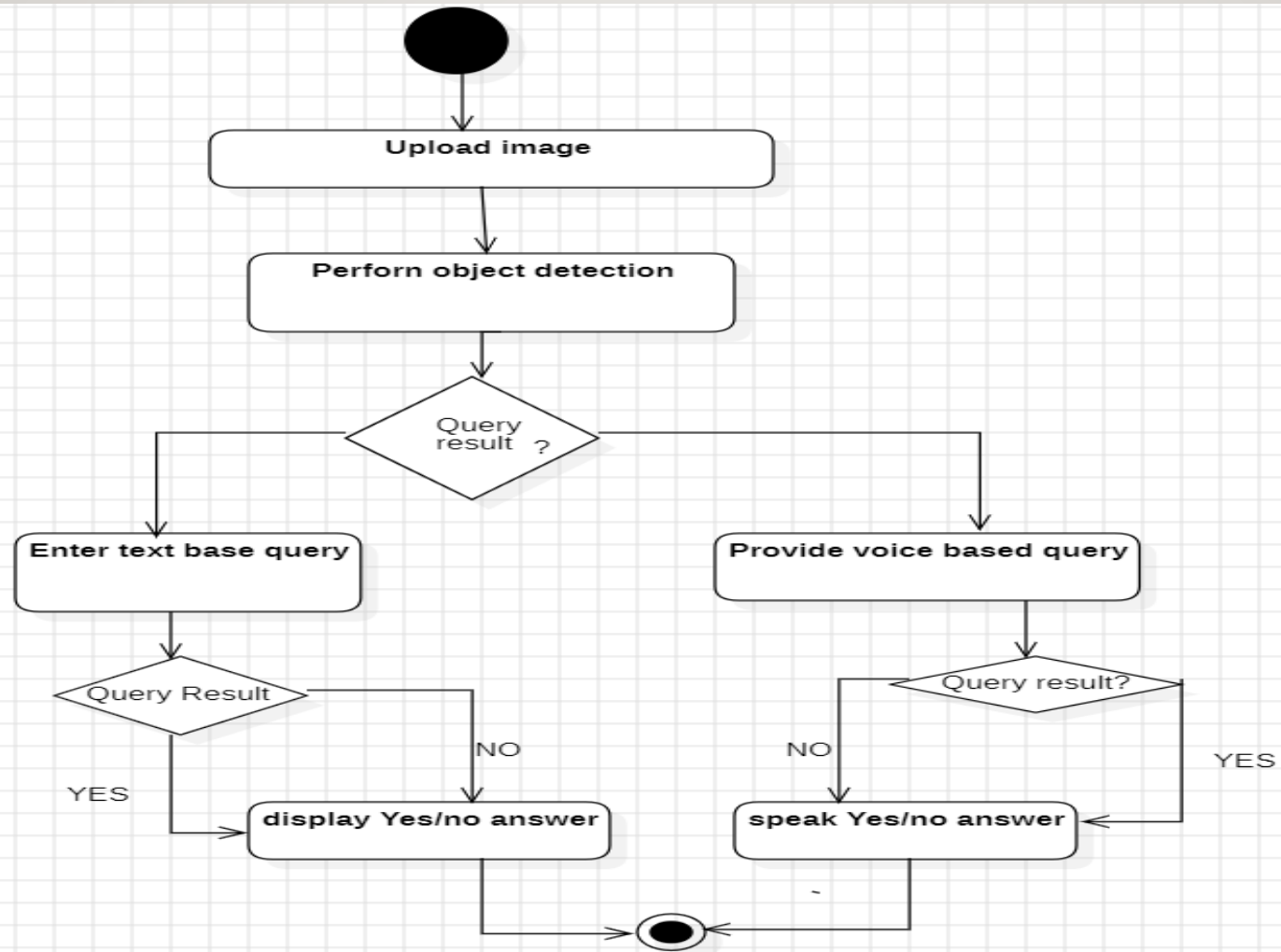
USE CASE DIAGRAM:



SEQUENCE DIAGRAM:



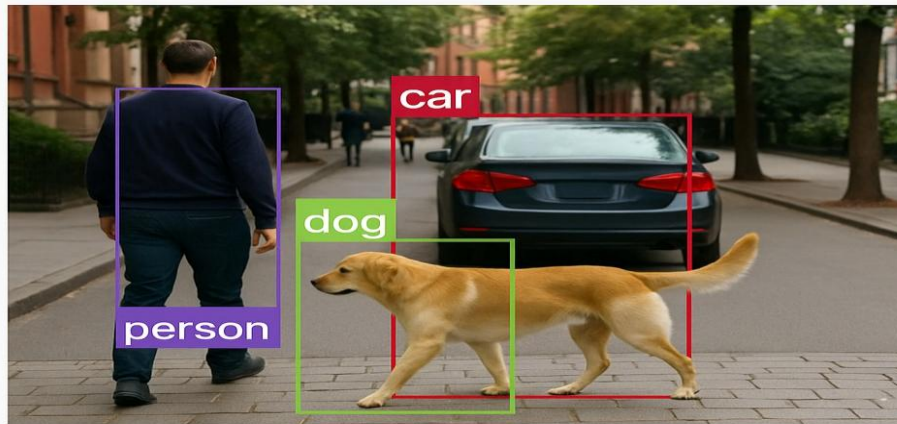
ACTIVITY DIAGRAM:



OUTPUT:

Real-Time Object Detection

Upload Image



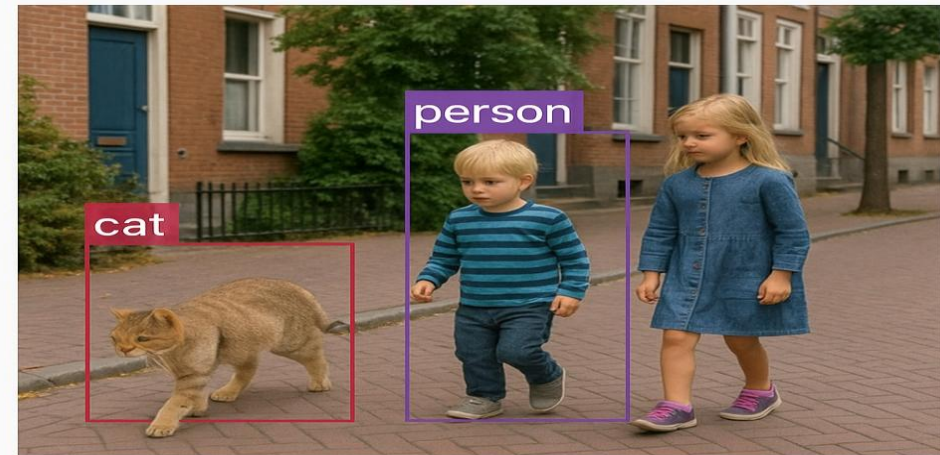
Is there a dog?



Yes, a dog is present.

Real-Time Object Detection

Upload Image



Is there a cat?



Yes, a cat is present.

CONCLUSION

- Developed a user-friendly web application for object detection using uploaded images.
- Integrated the SSD model with TensorFlow to ensure fast and accurate object detection.
- Used Flask, OpenCV, and front-end technologies to build a complete, responsive system.
- Enables real-time visual feedback with labeled bounding boxes for detected objects.
- Demonstrates the practical application of deep learning in a simple, accessible interface.
- Provides a foundation for further enhancements such as video detection or model training.

THANKYOU

